

UNIVERSIDAD DE GRANADA

PERIFÉRICOS Y DISPOSITIVOS DE INTERFAZ HUMANA
CURSO 2025/2026

Práctica 5. Experimentación con el sistema de salida de sonido

Autora:
Inés Ruiz Sánchez

15 de mayo de 2026



UNIVERSIDAD
DE GRANADA

Índice

1. Introducción y objetivos	2
2. Requisitos mínimos	2
2.1. Ejercicio 1	2
2.2. Ejercicio 2	4
2.3. Ejercicio 3	7
2.4. Ejercicio 4	8
2.5. Ejercicio 5	9
2.6. Ejercicio 6	10
3. Requisitos ampliados	11
4. Anexo	12
4.1. Fichero con requisitos mínimos	12

Índice de figuras

1. Convertir texto a sonido	2
2. Convertir texto a sonido	3
3. Descargar sonido	3
4. Fichero de sonido con apellido	4
5. Cargar y leer los ficheros de sonido	5
6. Cargar y leer los ficheros de sonido	6
7. Dibujo de fichero “nombre.wav”	6
8. Dibujo de fichero “apellidos.wav”	7
9. Cabeceras de los ficheros	8
10. Unión de dos sonidos	9
11. Gráfica de la onda unida	10
12. Creación de “basico.wav”	11

1. Introducción y objetivos

Los objetivos concretos de esta práctica son:

- Identificar y representar gráficamente la forma de onda de señales de sonido.
- Conocer la estructura de un fichero típico de sonido (ficheros WAV).
- Entender y operar con los parámetros principales de una señal de sonido.

En el anexo⁴ se incluyen los ficheros de códigos al completo.

Para obtener los ficheros WAV he usado la página <https://speechgen.io/>.

Repositorio en GitHub: <https://github.com/ruiz314/PDIH/tree/main/P5>.

Para poder trabajar con R y RStudio en mi ordenador personal realicé los seminarios y tutoriales proporcionados por el profesor. Se encuentran en el siguiente repositorio: <https://github.com/ruiz314/PDIH/tree/main/S-sonido>.

2. Requisitos mínimos

Para superar esta práctica se proponen los siguientes ejercicios, que se podrán realizar en un mismo programa:

- Crear dos ficheros de sonido (WAV) para realizar los siguientes ejercicios. En el primero debe escucharse el nombre de la persona que realiza la práctica. En el segundo debe escucharse el apellido.
- Leer los dos ficheros de sonido creados y dibujar la forma de onda de ambos sonidos (por separado).
- Obtener la información de las cabeceras de ambos sonidos.
- Unir ambos sonidos en uno nuevo para escuchar el nombre y apellido correctamente.
- Dibujar la forma de onda de la señal y reproducir el sonido resultante (una vez unidos).
- Almacenar el sonido resultante en un archivo nuevo llamado “basico.wav”

2.1. Ejercicio 1

Para crear los ficheros de sonido en la página <https://speechgen.io/> hay que seleccionar el idioma y la voz (arriba a la izquierda). A continuación hay que escribir el texto en el recuadro. Observar Figura 1.

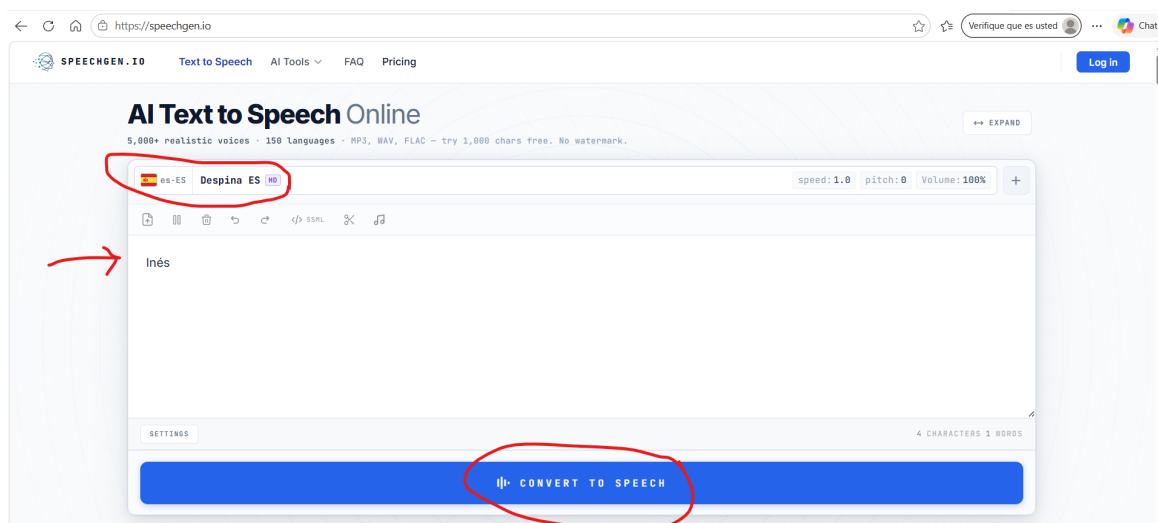


Figura 1: Convertir texto a sonido

Abajo a la izquierda hay un botón de “SETTINGS” (Figura 2) que sirve para cambiar la configuración del archivo.

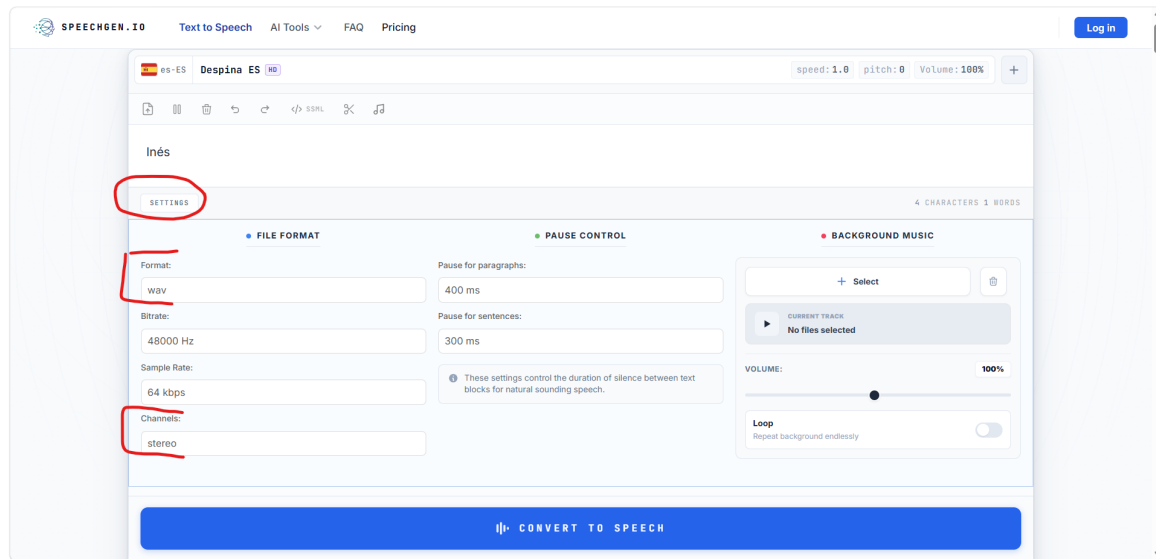


Figura 2: Convertir texto a sonido

Seleccionar formato “wav”.

Por último clicar en “CONVERT TO SPEECH”. Aparecerá abajo el audio, y podemos descargarlo pulsando el botón “DOWNLOAD”. Observar Figura 3.

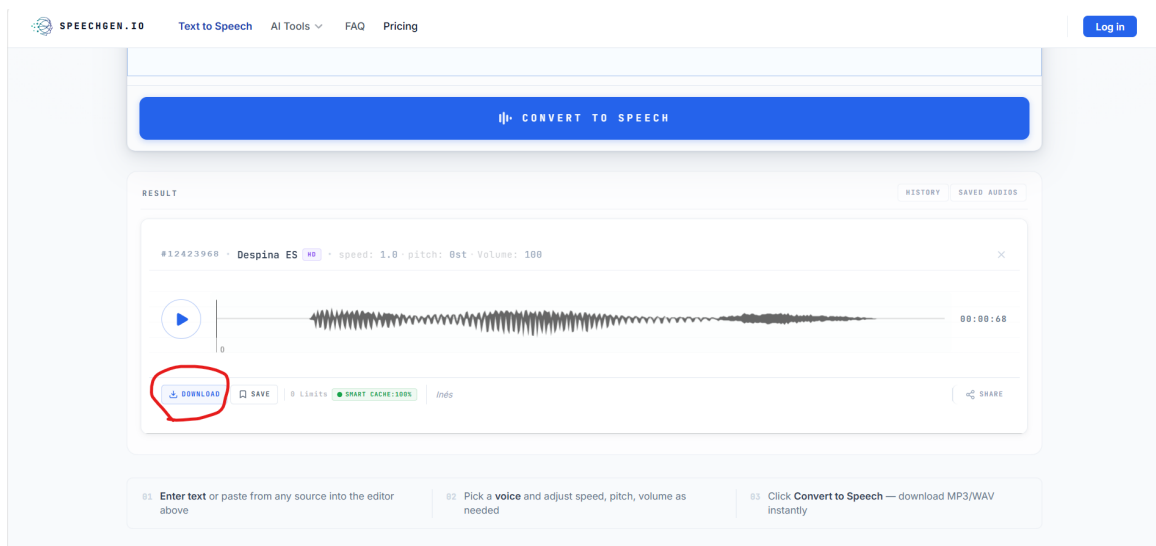


Figura 3: Descargar sonido

Para crear el fichero de sonido con los apellidos hay que realizar el mismo procedimiento. Observar Figura 4.

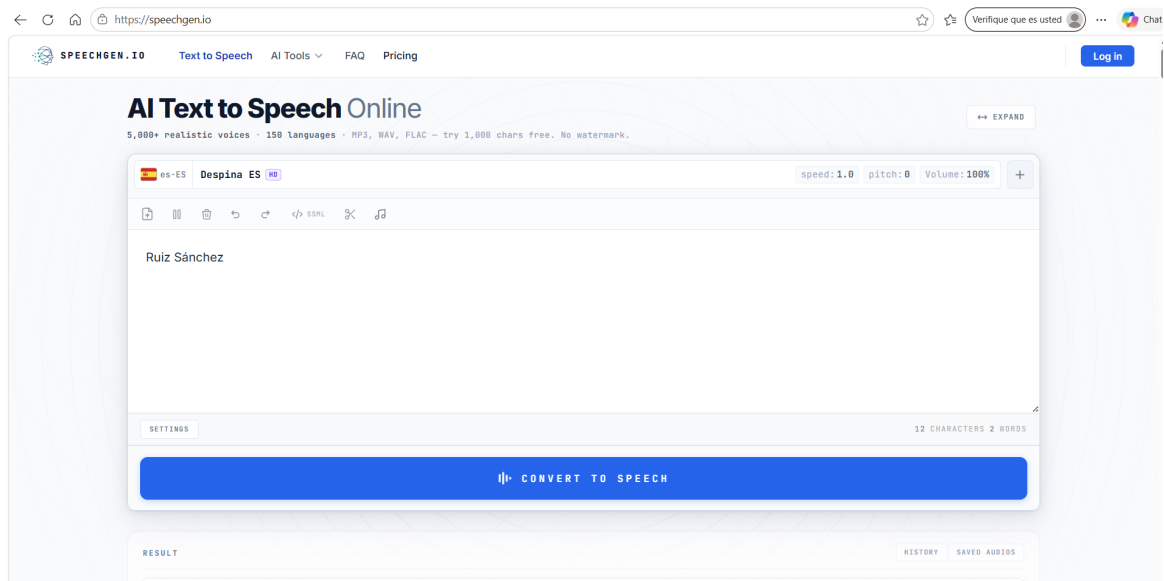


Figura 4: Fichero de sonido con apellido

2.2. Ejercicio 2

Para cargar los ficheros de audio se usa la función *readWave()*:

```
1 # Audio con el nombre
2 nombre <- readWave('nombre.wav')
3 nombre
4
5 # Audio con los apellidos
6 apellidos <- readWave('apellidos.wav')
7 apellidos
```

Listing 1: Cargar los ficheros de sonido en R

Llamar al fichero de sonido (líneas 3 y 7) hace que se muestre información como el número de muestras, la duración y el número de canales. Observar Figura 5.

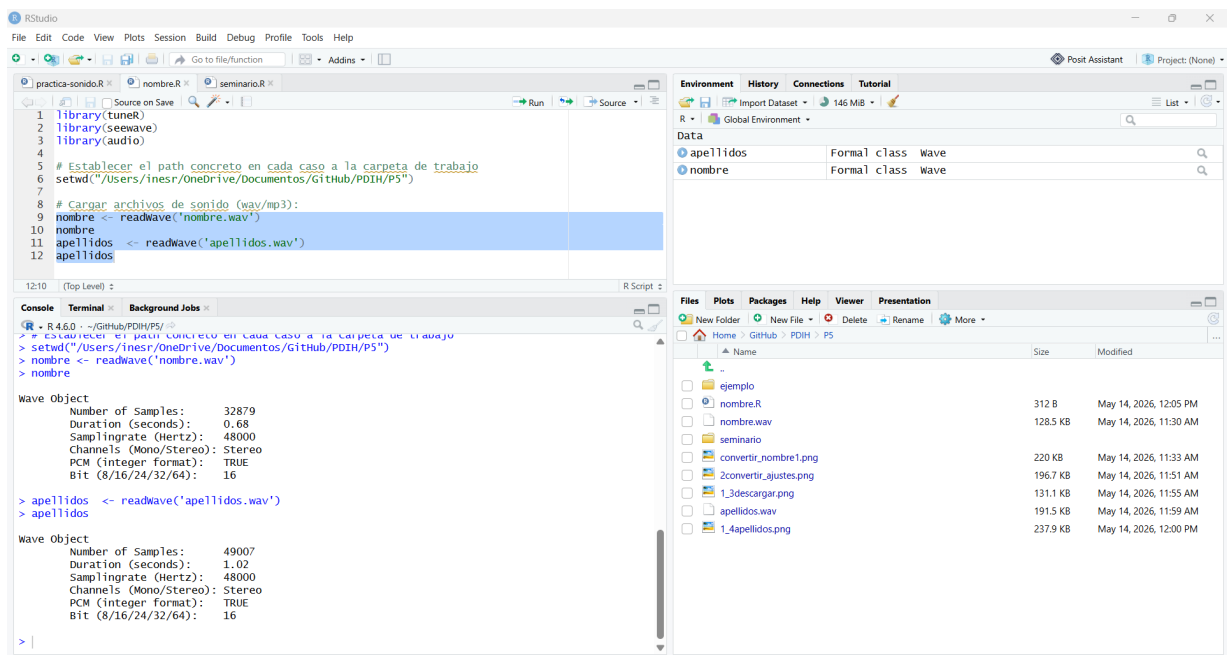


Figura 5: Cargar y leer los ficheros de sonido

Los datos obtenidos son los siguientes (ver Cuadro 1):

Cuadro 1: Información de los objetos Wave: *nombre* y *apellidos*

Parámetro	Objeto: <i>nombre</i>	Objeto: <i>apellidos</i>
Número de muestras	32879	49007
Duración (segundos)	0.68	1.02
Tasa de muestreo (Hz)	48000	48000
Canales	Stereo	Stereo
Formato PCM	TRUE	TRUE
Bit	16	16

Para obtener el dibujo de la onda la función *plot* necesita saber el número de muestras que tiene el fichero. Observamos que el fichero *nombre* tiene 32879 muestras y el fichero *apellidos* tiene 49007.

```

1 # Mostrar la onda del sonido
2 plot( extractWave(nombre, from = 1, to = 32879) )
3
4 plot( extractWave(apellidos, from = 1, to = 49007) )

```

Listing 2: Comandos para dibujar las ondas

Al usar la función *plot* obtenemos la gráfica que se muestra en la Figura 6:

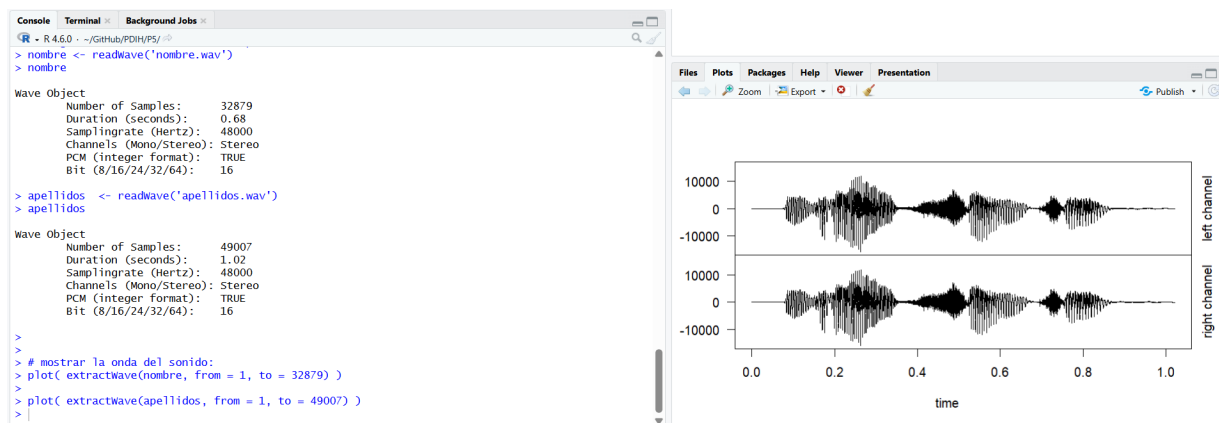


Figura 6: Cargar y leer los ficheros de sonido

El dibujo de la onda del archivo de sonido “nombre.wav” es el que se muestra en la Figura 7. El dibujo obtenido para el sonido de “apellidos.wav” se observa en la Figura 8.

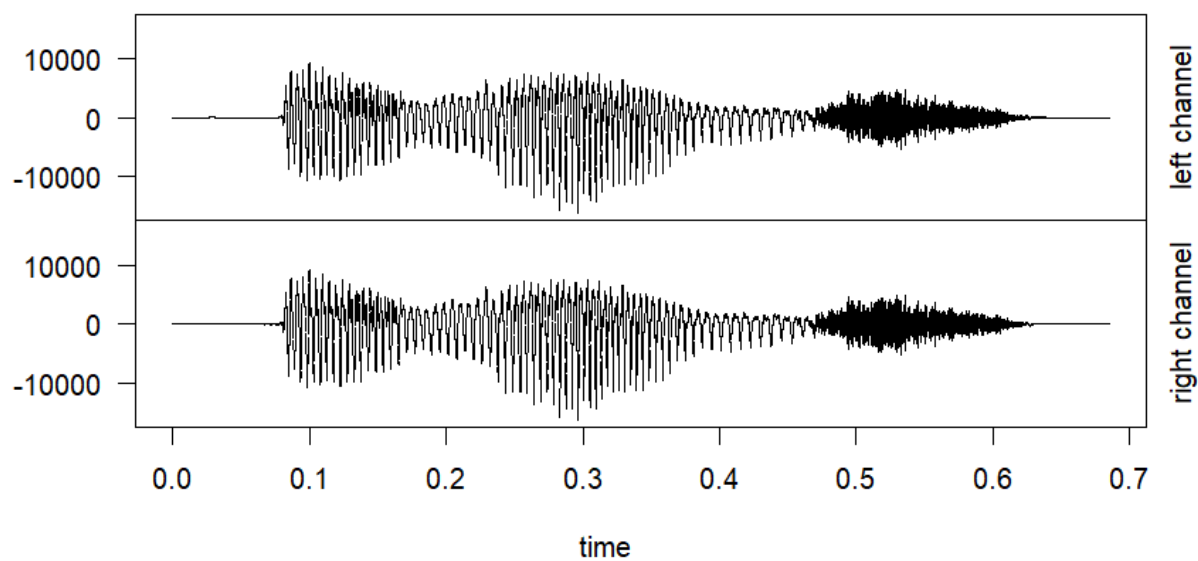


Figura 7: Dibujo de fichero “nombre.wav”

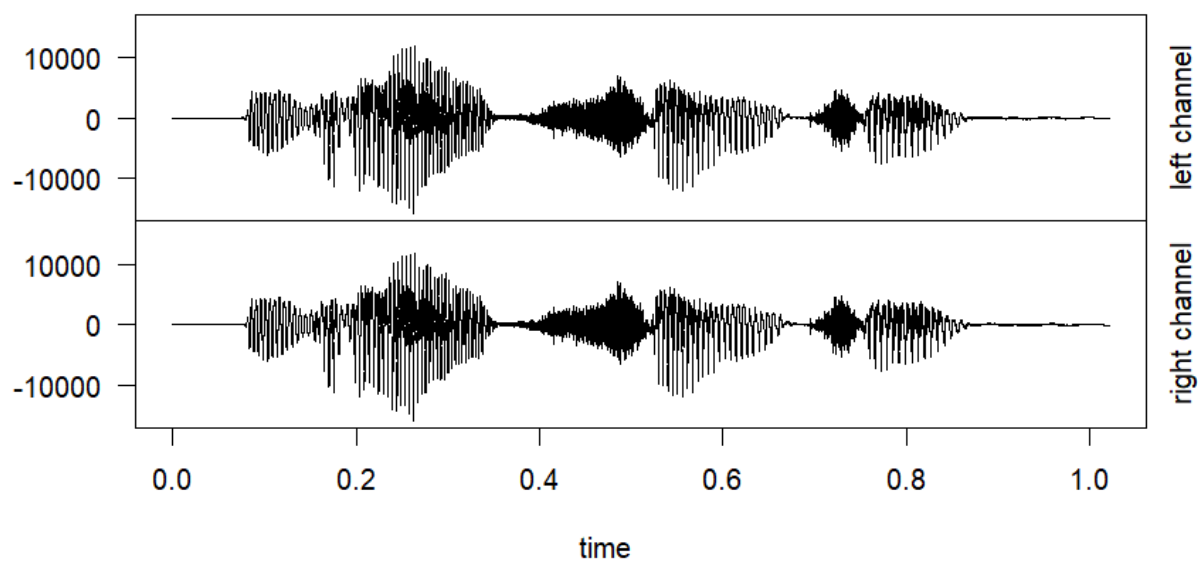


Figura 8: Dibujo de fichero “apellidos.wav”

2.3. Ejercicio 3

Usando la función *str()* se puede inspeccionar las cabeceras de archivos de sonido.

```
1 str(nombre)
2 str(apellidos)
```

Listing 3: Funciones para las cabeceras de los ficheros de sonido

En la siguiente imagen se ve la información obtenida (Figura 9).


```

10 # onda de ambos sonidos (por separado).
11
12 # Cargar archivos de sonido (wav):
13 nombre <- readWave('nombre.wav')
14 nombre
15 apellidos <- readWave('apellidos.wav')
16 apellidos
17
18
19 # Mostrar la onda del sonido:
20 plot( extractWave(nombre, from = 1, to = 32879) )
21 plot( extractWave(apellidos, from = 1, to = 49007) )
22
23 # Ejercicio 3. Obtener la información de las cabeceras de ambos sonidos.
24 str(nombre)
25 str(apellidos)
26
27

```

```

> plot( extractWave(nombre, from = 1, to = 32879) )
> plot( extractWave(apellidos, from = 1, to = 49007) )
> str(nombre)
Formal class 'Wave' [package "tuner"] with 6 slots
..@ left      : int [1:32879] 0 0 0 0 0 0 0 0 0 ...
..@ right     : int [1:32879] 0 0 0 0 0 0 0 0 0 ...
..@ stereo    : logi TRUE
..@ samp.rate : int 48000
..@ bit       : int 16
..@ pcm       : logi TRUE
> str(apellidos)
Formal class 'Wave' [package "tuner"] with 6 slots
..@ left      : int [1:49007] 0 0 0 0 0 0 0 0 0 ...
..@ right     : int [1:49007] 0 0 0 0 0 0 0 0 0 ...
..@ stereo    : logi TRUE
..@ samp.rate : int 48000
..@ bit       : int 16
..@ pcm       : logi TRUE
>

```

Figura 9: Cabeceras de los ficheros

Como ambos sonidos tienen dos canales, el atributo *left* y el atributo *right* valen lo mismo para cada sonido. Además, confirmamos que tienen dos canales porque la variable *stereo* es verdadera.

2.4. Ejercicio 4

Con la función *pastew()* se unen dos sonidos. Hay que prestar atención al orden, pues el primer argumento será el segundo sonido que se escuche y el segundo argumento será el primer sonido que se escuche.

```

1 union <- pastew(apellidos, nombre, output="Wave")

```

Listing 4: Función para unir dos sonidos

En la Figura 10 se puede observar la información del sonido.

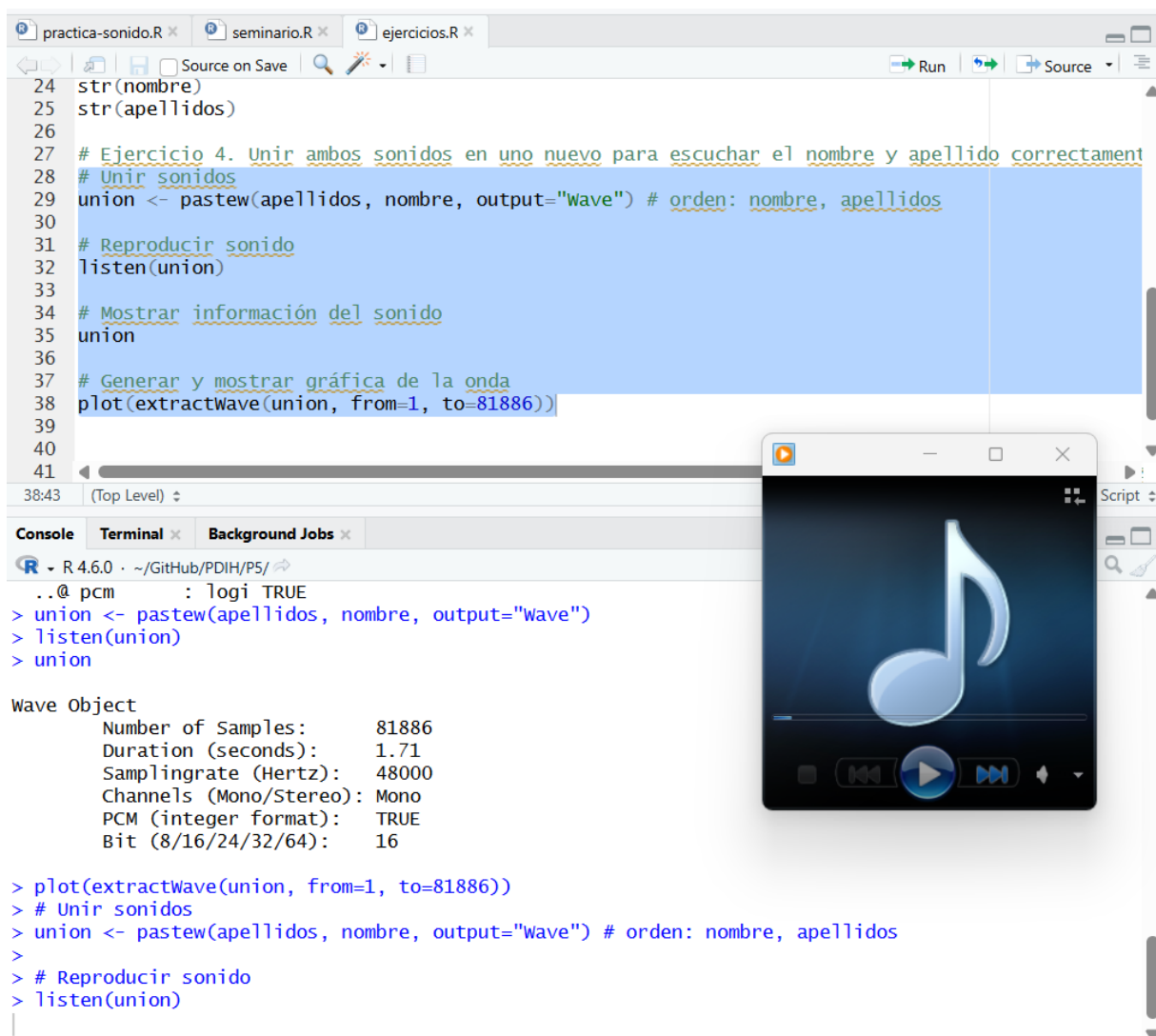


Figura 10: Unión de dos sonidos

En el Cuadro 2 se pueden observar los datos de los tres ficheros. La duración del fichero *union* es la suma de las duraciones del fichero *nombre* y el fichero *apellidos*, aproximadamente. Sucede lo mismo con el número de muestras: $32,879 + 49,007 = 81,886$.

Cuadro 2: Comparativa de los objetos Wave: *nombre*, *apellidos* y *union*

Parámetro	Objeto: nombre	Objeto: apellidos	Objeto: union
Número de muestras	32879	49007	81886
Duración (segundos)	0.68	1.02	1.71
Tasa de muestreo (Hz)	48000	48000	48000
Canales	Stereo	Stereo	Mono
Formato PCM	TRUE	TRUE	TRUE
Bit	16	16	16

2.5. Ejercicio 5

Llamando a la función `plot()` se genera una imagen con la onda resultante (Observar la Figura 11).

```

1 # Generar y mostrar dibujo de la onda
2 plot(extractWave(union, from=1, to=81886))
3

```

```

4 # Reproducir sonido
5 listen(union)

```

Listing 5: Generación de gráfica de unión

Además, para reproducir el sonido nuevo generado se usa la función `listen(union)` y abre el reproductor en el ordenador. Se puede observar cómo se abre en la Figura 10.

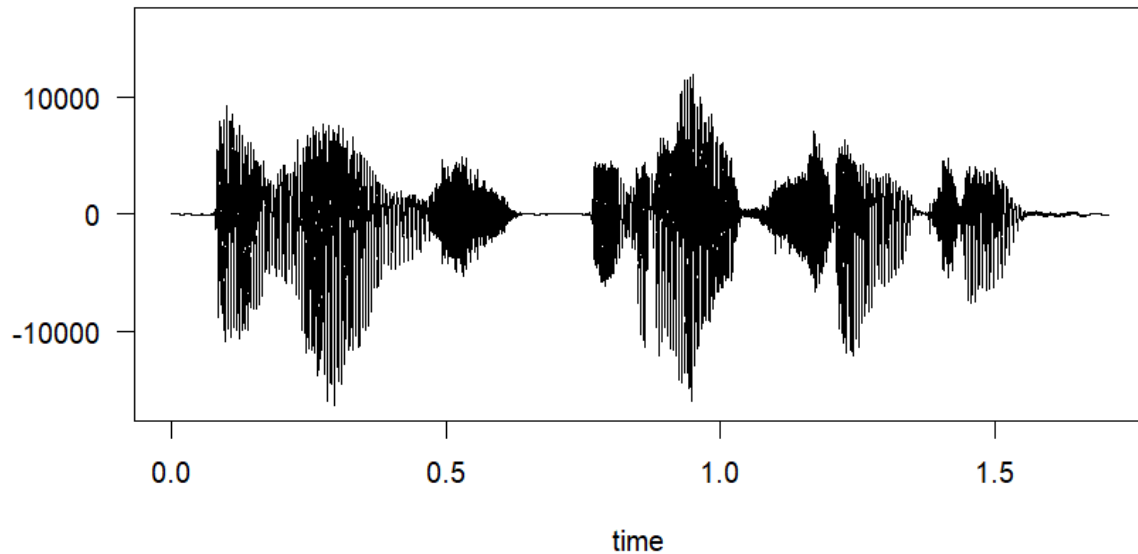


Figura 11: Gráfica de la onda unida

2.6. Ejercicio 6

Para guardar en disco el nuevo sonido generado se llama a la función `writeWave()` indicándole el sonido, y el nombre del fichero:

```

1 writeWave(union, file.path("basico.wav"))
2 basico <- readWave('basico.wav')
3 basico

```

Listing 6: Almacenar sonido union en fichero “basico.wav”

A continuación, en la Figura 12 se puede observar que la llamada a la función no genera ningún error y que el sonido ha sido almacenado con éxito. Además, se muestran los datos de la onda y coinciden con los datos de la onda *union*.

The screenshot shows an RStudio interface with three open files: `practica-sonido.R`, `seminario.R`, and `ejercicios.R`. The `ejercicios.R` file is active, displaying R code for audio processing. The code includes comments in Spanish and uses the `seewave` package functions. The console at the bottom shows the execution of the last two lines of code, resulting in a `Wave Object` with the following properties:

```
Wave Object
  Number of Samples: 81886
  Duration (seconds): 1.71
  Samplingrate (Hertz): 48000
  Channels (Mono/Stereo): Mono
  PCM (integer format): TRUE
  Bit (8/16/24/32/64): 16
```

Figura 12: Creación de “basico.wav”

3. Requisitos ampliados

Como requisitos ampliados (opcionales para subir nota) se propone:

- Pasarle un filtro de frecuencia para eliminar las frecuencias entre 10.000Hz y 20.000Hz. Almacenar la señal obtenida como un fichero WAV denominado “filtrado.wav”
- Tomar el sonido que se creó antes (lo tendremos en el archivo llamado “basico.wav”) para aplicarle el efecto de eco. Guardar ese sonido en un archivo nuevo llamado “eco.wav”. A continuación, se le debe dar la vuelta al sonido y almacenarlo como un fichero llamado “alreves.wav”

4. Anexo

4.1. Fichero con requisitos mínimos

```
1 # PDIH. Práctica 5
2 library(tuneR)
3 library(seewave)
4 library(audio)
5
6 # Establecer el path concreto a la carpeta de trabajo
7 setwd("/Users/inesr/OneDrive/Documentos/GitHub/PDIH/P5")
8
9 # Ejercicio 2. Leer los dos ficheros de sonido creados y dibujar la forma de
10 # onda de ambos sonidos (por separado).
11
12 # Cargar archivos de sonido (wav):
13 nombre <- readWave('nombre.wav')
14 nombre
15 apellidos <- readWave('apellidos.wav')
16 apellidos
17
18
19 # Mostrar la onda del sonido:
20 plot( extractWave(nombre, from = 1, to = 32879) )
21 plot( extractWave(apellidos, from = 1, to = 49007) )
22
23 # Ejercicio 3. Obtener la información de las cabeceras de ambos sonidos.
24 str(nombre)
25 str(apellidos)
26
27 # Ejercicio 4. Unir ambos sonidos en uno nuevo para escuchar el nombre y apellido
28 # correctamente.
29 # Unir sonidos
30 union <- pastew(apellidos, nombre, output="Wave") # orden: nombre, apellidos
31
32 # Mostrar información del sonido
33 union
34
35 # Ejercicio 5. Dibujar la forma de onda de la señal y reproducir el sonido resultante
36 # (una vez unidos).
37 # Generar y mostrar dibujo de la onda
38 plot(extractWave(union, from=1, to=81886))
39
40 # Reproducir sonido
41 listen(union)
42
43 # Ejercicio 6. Almacenar el sonido resultante en un archivo nuevo llamado "basico.wav"
44 "
45 writeWave(union, file.path("basico.wav"))
46 basico <- readWave('basico.wav')
47 basico
```